Lab 3: Designing and Simulating an Arithmetic Logic Unit (ALU)

In previous labs (Lab 1 and Lab 2), you designed an adder, a shifter/rotator, and a multiplier. In this lab, you will design a comparator and integrate all these components, along with a multiplexer and demultiplexer, to create a complete Arithmetic Logic Unit (ALU).

A. Comparator

Implement the $Log_2(N)$ parallel comparator to compare two 4-bit unsigned numbers

Hint: Refer to page 90 of Lecture 7 for additional guidance.

B. ALU

An arithmetic logic unit (ALU) can perform four type of functions: Arithmetic operations, logic operations, comparison, and shifting/rotating operations.
The ALU has unsigned 8-bit data inputs, X and Y. It has a 16-bit unsigned data output called Z. To select among different functions, the ALU also provides two 2-bit control signals. They are S and M.
The functions are defined by the following table:

S	M	Function	Function Category
00	00	ADD	Arithmetic
	01	SUB	
	10	MULTIPLY	
	11	UNUSED	
01	00	ROTATE RIGHT	Shift/Rotate
	01	ROTATE LEFT	
	10	SHIFT RIGHT	
	11	SHIFT LEFT	
10	00	GREATER THAN	Compare
	01	LESS THAN	
	10	EQUAL	
	11	NOT EQUAL	
11	00	AND	Logic
	01	OR	
	10	NOT	
	11	XOR	

You are required to implement the following modules:

- **Arithmetic module**: This module should include the adder, subtractor, and multiplier functions.
- **Shifter/Rotator module**: Responsible for implementing the shifting and rotating operations.
- **Comparator module**: Implements comparison functions like greater than, less than, equal to, and not equal to.
- **Logic module**: Implements the logical operations such as AND, OR, NOT, and XOR.
- **ALU module**: Integrates the Arithmetic, Shifter/Rotator, Comparator, and Logic modules to perform all required ALU functions.
  You may also need to incorporate other combinational blocks, such as multiplexers, demultiplexers, or additional logic gates.

```
module alu(
        input logic [7:0] x,
        input logic [7:0] y,
        input logic [1:0] m,
        input logic [1:0] s,
        output logic[15:0] z);

//
endmodule
```

Testbench Requirements:

1. Create a testbench to verify the functionality of the ALU.
2. The testbench should validate all possible ALU functions.
3. Use randomized inputs to test your design thoroughly.

Hint: Refer to page 116 of Lecture 7 for additional guidance.

C. (Bonus) Implement the ALU on FPGA.

You can implement a 4-bit version of the ALU on an FPGA as a bonus task. Connect the two 4-bit inputs, X and Y, to the FPGA board's switches (SW), where each bit of X and Y corresponds to an individual switch. The 2-bit control signals, S and M, should also be connected to switches (SW) to allow manual selection of the ALU's operations. The 8-bit output of the ALU should be connected to LEDs on the board, with each LED representing one bit of the binary output. Additionally, the binary values of the inputs, X and Y, should also be displayed using LEDs.

You need to show a schematic of your design during your Lab presentation.